

DESARROLLO DE INTERFACES



UT5-Documentar una Aplicación

DOCUMENTAR UNA APLICACIÓN

Cuando se habla de **documentación de aplicaciones** software nos referimos a todos los **elementos que detallan y aclaran las características de una aplicación** y que pueden ser necesarios para su uso, modificación.

Se trata de **ficheros de ayuda, manuales** y demás **guías de uso y/o mantenimiento** ya sean **dirigidas para los usuarios** de la aplicación, para equipo de soporte **o incluso para otros desarrolladores, diseñadores.**

DOCUMENTAR UNA APLICACIÓN

En un sistema software es **especialmente importante** contar con una **documentación completa** y además **sencilla de consultar**. Los motivos son varios:

Facilitar su comprensión y posibilitar su uso por parte del usuario independientemente de cuál sea su perfil

- Facilitar el **mantenimiento** de la aplicación
- Facilitar el **desarrollo** de mejoras para otros equipos de trabajo
- Facilitar **que el sistema pueda ser comprendido** por otros desarrolladores /diseñadores de software

DOCUMENTAR UNA APLICACIÓN

No es suficiente con desarrollar aplicaciones funcionales y atractivas, sino que, además, estas deben contar con una **documentación adecuada**.

Documentar una aplicación es una tarea **tan importante como la de realizar el propio código**.

Utiliza **Javadoc** para generar documentación técnica de tu código. Es una herramienta estándar en Java que **permite generar documentación HTML a partir de comentarios en el código fuente**.

TIPOS DE DOCUMENTACIÓN

A grandes rasgos podemos dividir la documentación de aplicaciones de software en **dos grupos**: **documentación de pruebas** y **documentación técnica**

Documentar las pruebas realizadas sobre un programa determinado es fundamental para **detectar y corregir posibles errores que** podemos distinguir a su vez **dos tipos**

A -Documentación de entrada: en la que se especifican los escenarios de prueba y se detallan los procedimientos de las mismas.

B -Documentación de salida: se trata de los informes resultantes de aplicar las pruebas sobre el programa definidas en el punto anterior.

TIPOS DE DOCUMENTACIÓN

Documentación Técnica pertenece a este grupo el resto de documentación: Guías, hojas de especificaciones, manuales, al igual que en el caso anterior se pueden dividir en dos grupos.

A-Documentación Interna. Orientada a los equipos de desarrollo.

B-Documentación externa. Orientada a los usuarios finales de la aplicación

TIPOS DE DOCUMENTACIÓN

Documentación Interna. Su propósito es facilitar el mantenimiento, la evolución del software, y comprensión técnica de la aplicación

DOCUMENTACIÓN DE DISEÑO (Arquitectura y Análisis)

Diagramas de Clases, de Secuencia, De uso, De estado, De Despliegue, de Componentes, Modelo E-R, etc

DOCUMENTACIÓN DE CÓDIGO (Código del proyecto) JavaDoc, API, Configuración, Dependencias.

DOCUMENTACIÓN DE CONFIGURACIÓN Y DEPENDENCIAS: Archivo README técnico Gestión de dependencias (Maven, Gradle, npm, etc.) Variables de entorno y configuración Documentación del entorno de desarrollo (IDE, frameworks, versiones)

TIPOS DE DOCUMENTACIÓN

Documentación Externa. Orientada al usuario final, no necesariamente programadores., técnicos de soporte, usuarios de la aplicación.

MANUAL TÉCNICO (Personal de Soporte)

En él se incluye información para instalar, configurar y desplegar la aplicación

MANUAL DE USUARIO (Usuario Final)

En el se explica como usar la aplicación, se incluyen capturas de pantalla y ejemplos de como se realizan las diferentes funcionalidades de la aplicación.



TIPOS DE DOCUMENTACIÓN

Documentación interna: *cómo está hecha por dentro.* Incluye **código, diagramas, diseño, arquitectura** y documentación necesaria para que otros desarrolladores hagan evolutivos.

Documentación externa: *cómo **usar, instalar, configurar y administrar la aplicación.***

Documentación Interna

Documentando el código

Es muy útil fijarse en el formato que tiene la documentación de métodos y constructores ya existentes en la **API de Java**. Basta con situar el cursor del ratón encima de algún método o constructor.

docs.oracle.com/en/java/javase/25/docs/api/index.html

OVERVIEW TREE PREVIEW NEW DEPRECATED INDEX SEARCH HELP Java SE 25 & JDK 25

String

Java® Platform, Standard Version 25 API Specification

This document is divided into two sections:

- Java SE
The Java Platform, Standard Edition, with java.
- JDK
The Java Development Kit (JDK) modules whose names start with j.

Classes and Interfaces

String	Class in package java.lang
StringBuffer	Class in package java.lang
StringBuilder	Class in package java.lang
StringJoiner	Class in package java.util
StringReader	Class in package java.io
StringTokenizer	Class in package java.util
StringWriter	Class in package java.io
AttributedString	Class in package java.text
StringContent	Class in package javax.swing.text



Documentación Interna

Documentando el código

Cabeceras de Clase

Cada clase debe incluir la **documentación que describa su propósito y su relación con otras clases.**

Documentación Interna

Documentando el código

```
/**
 * Clase Cuenta.
 *
 * Representa una cuenta bancaria con operaciones básicas como ingreso
 * y retirada de saldo. Esta clase se relaciona con la clase Cliente,
 * ya que cada cuenta pertenece a un cliente.
 *
 * @author Pepe Perez
 * @version 1.0
 * @since 2023-12-24
 * @see Cliente
 */
public class Cuenta {

}
```

Ejemplo de cabecera de clase

Cabeceras de Clase

Cada clase debe incluir la documentación que describa su propósito y su relación con otras clases.

Documentación Interna

Documentando el código

Cabeceras de Método: Proporciona una descripción clara de cada método incluyendo:

- **Parámetros de entrada** (@param).
- **Valor devuelto** (@return).
- **Excepciones lanzadas** (@throws), si aplica.

Documentación Interna

Documentando el código

Ejemplo de cabecera de método

```
/**
 * Calcula el área de un rectángulo.
 *
 * @param ancho El ancho del rectángulo en centímetros.
 * @param alto La altura del rectángulo en centímetros.
 * @return El área resultante en centímetros cuadrados.
 * @throws IllegalArgumentException Si ancho o alto son negativos.
 */
public double calcularArea(double ancho, double alto) {
    if (ancho < 0 || alto < 0) {
        throw new IllegalArgumentException("Las medidas no pueden ser negativas");
    }
    return ancho * alto;
}
```

Documentación Interna

Ejemplo de cabecera de método

```
/**
 * Calcula el interés acumulado de una cuenta bancaria.
 *
 * @param tasaInteres Tasa de interés anual en formato decimal (ejemplo: 0.05).
 * @param anios Número de años.
 * @return El interés acumulado como un valor decimal.
 * @throws IllegalArgumentException Si la tasa de interés o los años son negativos.
 */

public double calcularInteres(double tasaInteres, int anios) throws
IllegalArgumentException {
    // Código del método
}
```

Documentación Interna

Documentando el código

Cabeceras de Atributos: Descripción de los atributos de la clase, si son esenciales para comprender el comportamiento de la clase

```
/**
 * Nombre del titular de la cuenta.
 */
private String titular;

/**
 * Saldo actual de la cuenta bancaria.
 */
private double saldo;
```

“Se recomienda documentar atributos **solo cuando su propósito no sea obvio por el nombre** o cuando representen información **clave para entender el funcionamiento de la clase.**”

Documentación Interna

Documentando el código

Organización del Código: Organiza el código en paquetes.

Documenta el propósito de cada paquete utilizando un archivo **package-info.java**.

```
/**  
 * Este paquete contiene las clases relacionadas con la gestión de usuarios.  
 */  
package com.banco.gestionusuarios;
```

Documentación Interna

Documentando el código

Uso de Etiquetas estándar

@author: Indica el autor del código.

@version: Versión del código o clase.

@since: Versión desde la cual este código está disponible.

@deprecated: Marca código que no debe usarse más, con detalles de por qué y qué usar en su lugar.

@see: Proporciona referencias a clases o métodos relacionados.

@link y @linkplain: Para crear enlaces dentro de la documentación.

Documentación Interna

Documentando el código

Generar la documentación de un proyecto

```
javadoc -d ./docs -sourcepath ./src -subpackages com.miempresa
```

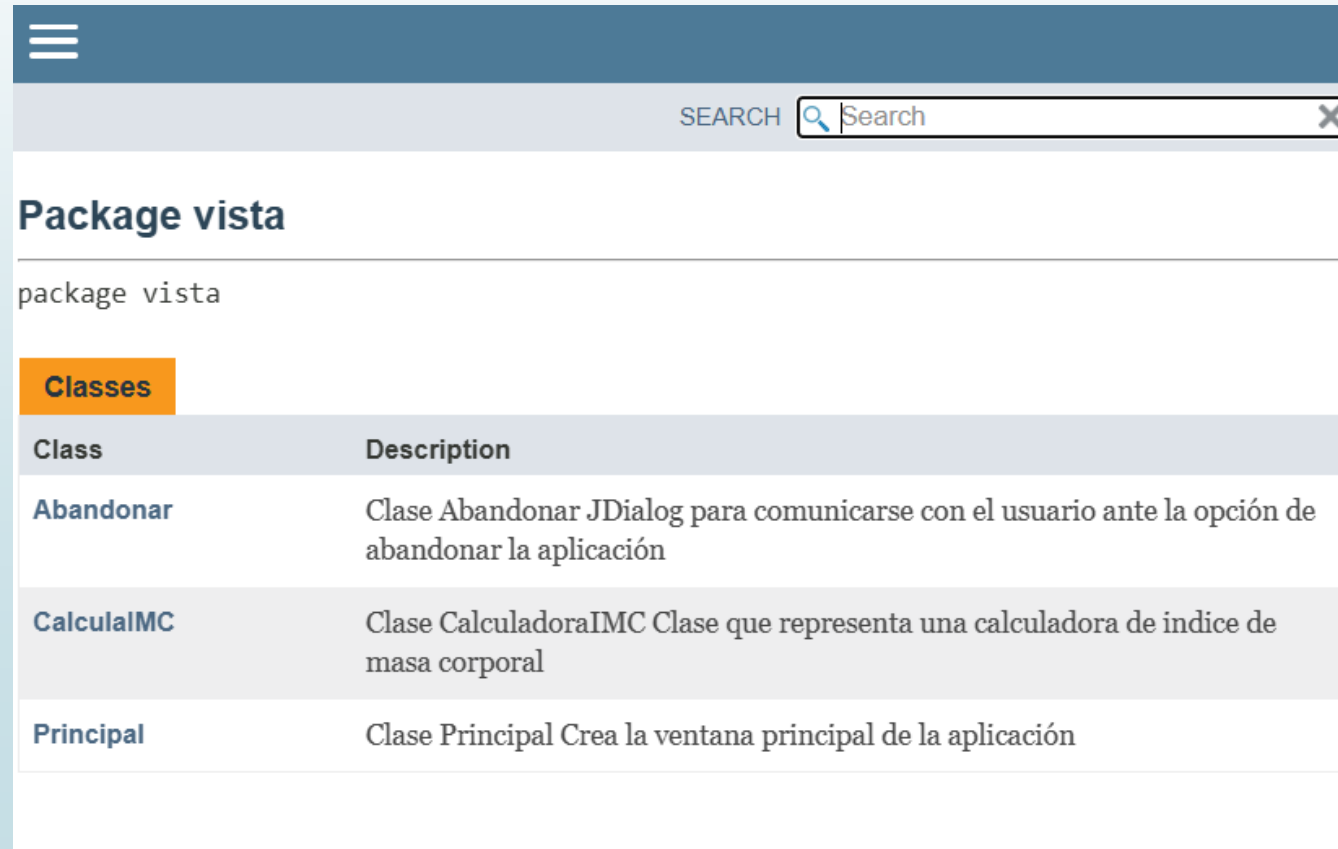
Se creará una carpeta denominada **docs**, donde se almacenará la documentación del proyecto. Basta con abrir index.html dentro de **docs** para navegar por la documentación.

Generar documentación con Maven: `mvn -q javadoc:javadoc`

Documentación Interna

Documentando el código

Documentación Generada



The screenshot shows a web interface for generated documentation. It features a dark blue header with a hamburger menu icon on the left and a search bar on the right. Below the header, the page title 'Package vista' is displayed. Underneath, the text 'package vista' is shown. A section titled 'Classes' is highlighted with an orange background. Below this, a table lists three classes: 'Abandonar', 'CalculaIMC', and 'Principal', each with a brief description.

Package vista	
package vista	
Classes	
Class	Description
Abandonar	Clase Abandonar JDialog para comunicarse con el usuario ante la opción de abandonar la aplicación
CalculaIMC	Clase CalculadoraIMC Clase que representa una calculadora de índice de masa corporal
Principal	Clase Principal Crea la ventana principal de la aplicación

Documentación Interna

Documentando el código

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos

En esta actividad deberás: **Documentar** clases, atributos y métodos con **Javadoc**.

Generar la documentación con **Maven** y con el **JDK**. Visualizar y validar el resultado en HTML.

Documentación Interna

Documentando el código

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos

Instrucciones

1. Abrir el proyecto

Descomprimir el ZIP. (Descargarlo previamente del **aula virtual**)

Importar como *Existing Maven Project* en Eclipse/IntelliJ, o trabajar desde la terminal.

2. Documentar

Añadir Javadoc a clases (`@author`, `@version`, `@since`, relaciones con `@see/{@link ... }`).

Documentar **métodos públicos** con `@param`, `@return`, `@throws`.

Documentar **atributos clave** si aportan claridad.

3. Generar documentación con Maven

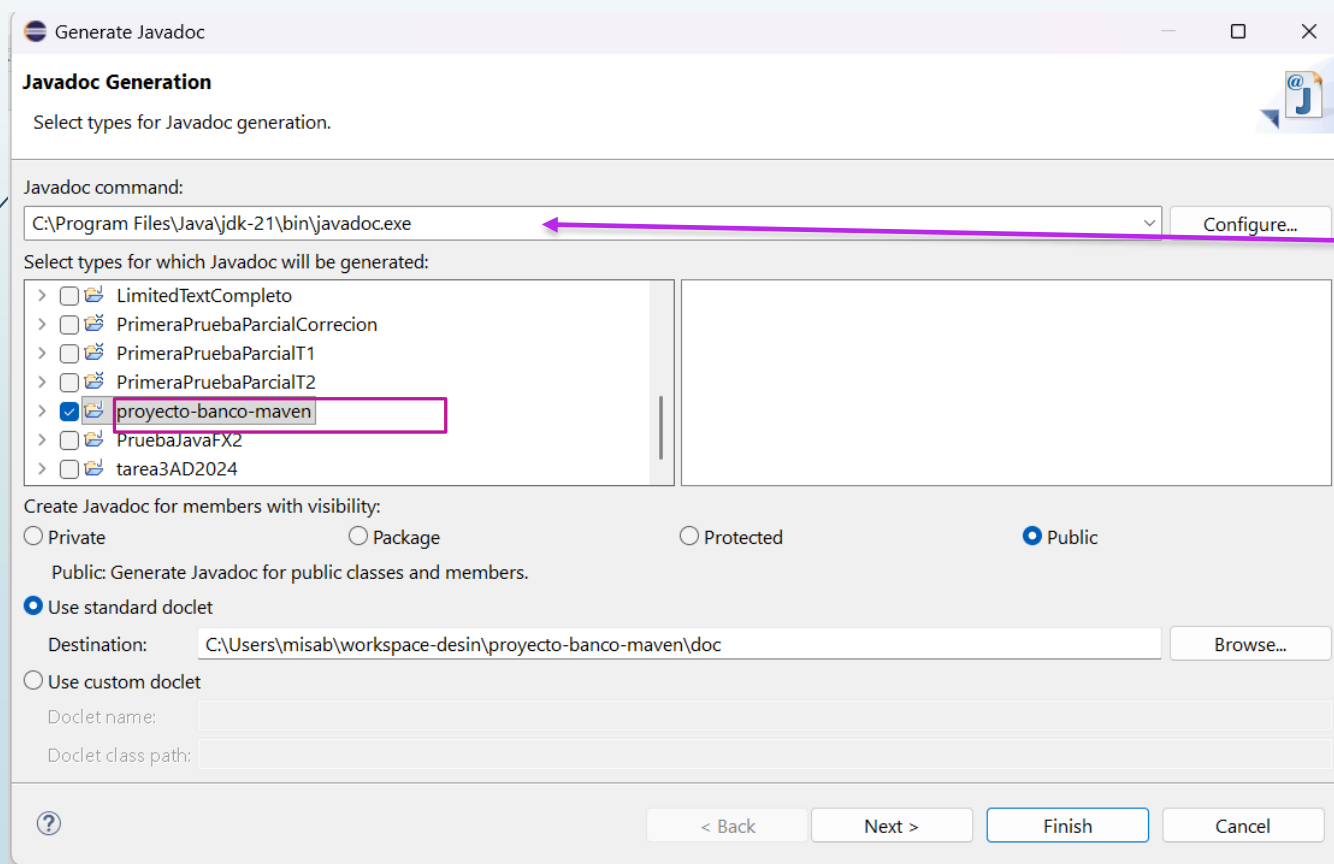
4. Comprobar documentación Generada

Documentación Interna

Documentando el código

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos



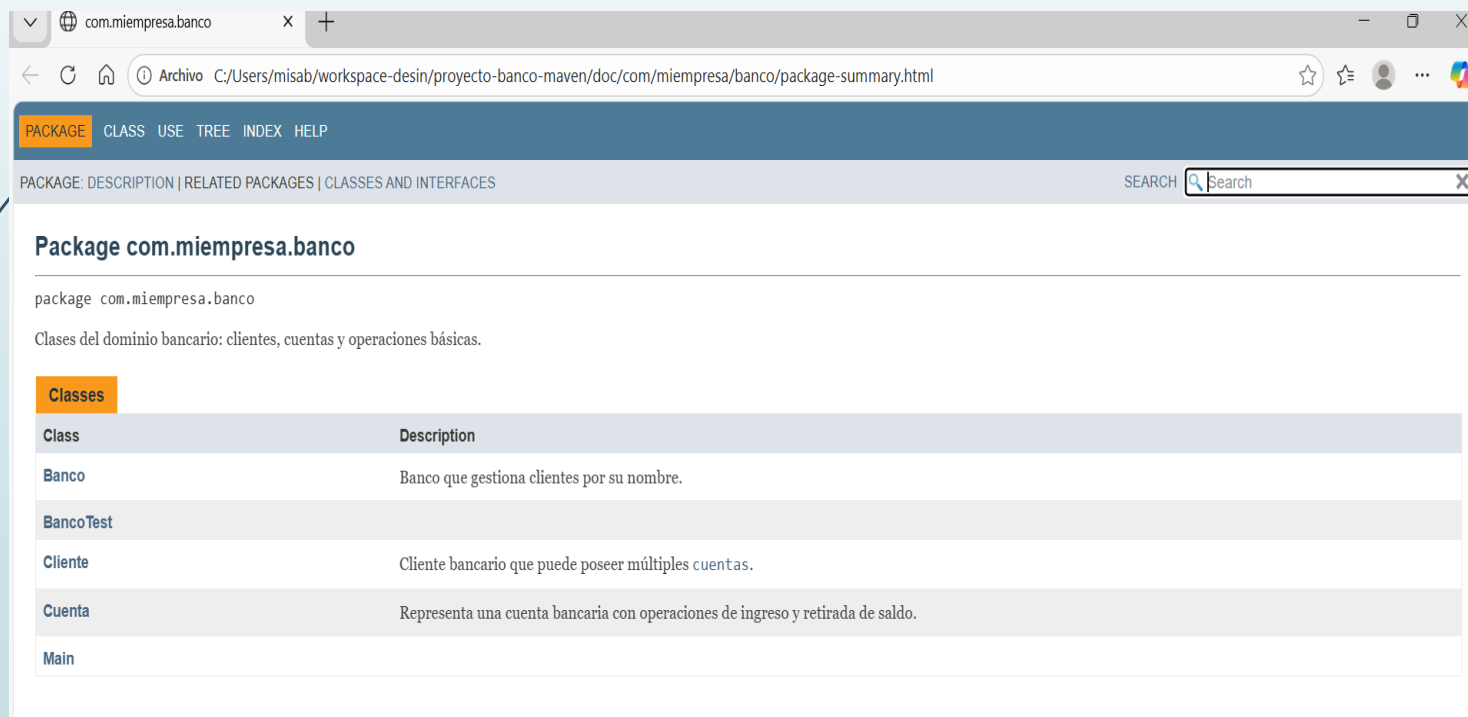
Seleccionar el **javadoc** de la jdk que corresponda, en este caso jdk21

Documentación Interna

Documentando el código

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos



PACKAGE CLASS USE TREE INDEX HELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES SEARCH

Package com.miestpresa.banco

package com.miestpresa.banco

Clases del dominio bancario: clientes, cuentas y operaciones básicas.

Classes

Class	Description
Banco	Banco que gestiona clientes por su nombre.
BancoTest	
Cliente	Cliente bancario que puede poseer múltiples cuentas.
Cuenta	Representa una cuenta bancaria con operaciones de ingreso y retirada de saldo.
Main	

Documentación Generada.

En este caso se ha generado sobre el proyecto base, no sobre el proyecto totalmente documentado

Documentación Interna

Documentando el código

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos

Entregar en el aula virtual la carpeta **docs** comprimida en formato zip

Documentación Interna

Documentación Técnica

Diagramas UML:

- **Diagrama de clases:** Relación y jerarquía entre clases.
- **Diagrama de casos de uso:** Para explicar las interacciones entre actores y funcionalidades.
- **Diagrama de secuencia:** Para detallar flujos de llamadas y eventos en procesos clave.
- **Diagrama de estados:** Útil si la aplicación tiene máquinas de estados significativas.
- **Esquema de arquitectura:**
 - Explicación de la estructura general de la aplicación: capas, servicios, patrones usados (MVC, DAO, etc.).
 - **Diagrama de paquetes** y módulos. Detallar la organización de la aplicación

Documentación Interna

Documentación Técnica

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos

Instrucciones

1.Instalar Visual Paradigm, herramienta para trabajar en UML



2.Haciendo uso de esta herramienta

Crear el diagrama de clases correspondiente a este proyecto

Crear el diagrama de casos de uso correspondiente al ingreso, retirada de dinero y creación de una cuenta

Crear el diagrama de estados de la clase cuenta.

3.Generar documento técnico que incluya: (Incluir Portada con datos del alumno, fecha e índice)

a) Objetivo del sistema

Un párrafo sobre para qué sirve el banco.

b) Arquitectura lógica (muy simple en este caso)

Capa de lógica de negocio

Capa de presentación (Main consola)

c) Especificación de clases

Documentación Externa

Manuales y Guías

Existen diferentes tipos de manuales y guías con diferentes formatos y soportes: papel, formato electrónico, web de ayuda, presentación, vídeo o una combinación de varios, según la complejidad de la aplicación y el perfil del usuario al que va dirigido.

Guía de Usuario

- Introducción y objetivos del software
- Requisitos mínimos del sistema
- Funcionalidades explicadas paso a paso con ejemplos

Documentación Externa

Manuales y Guías

Guía de Usuario. En formato papel, deberá contener

Portada

- **Título:** Nombre de la aplicación y versión
- **Subtítulo:** Una breve descripción del propósito del software (Ejemplo Guía de usuario para la gestión de contactos)
- **Logotipo o imagen representativa** (si aplica).
- **Autoría o equipo de desarrollo.**
- **Fecha de publicación y/o versión del documento.**
- **Tabla de Contenido**
 - Índice
 - Secciones principales

Documentación Externa

Manuales y Guías

Guía de Usuario. En formato papel, deberá contener las siguientes secciones

- **Introducción**, explicación de para qué sirve el documento y una breve descripción del público al que va dirigida la guía.
- **Requisitos**. Requisitos mínimos del sistema: Hardware, software, conexión a internet, etc.
- **Convenciones usadas en la guía** Uso de iconos, consejos, etc.
- **Guía básica de la aplicación**
- **Instalación y configuración**
 - Pasos detallados de la instalación
 - Pasos detallados de la configuración
- **Preguntas Frecuentes (FAQ)**

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

JavaHelp es una API diseñada para proporcionar sistemas de ayuda en aplicaciones Java. Está basado en HTML y XML

Aunque su uso ha disminuido con el tiempo debido a herramientas más modernas, sigue siendo una opción válida para integrar un sistema de ayuda profesional con navegación, búsqueda y contenido organizado en formato HTML.

A continuación, se detalla cómo crear un sistema de ayuda utilizando **JavaHelp**:

Última versión estable de JavaHelp, que deberás descargar

<https://central.sonatype.com/artifact/javahelp/javahelp>

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Última versión estable de JavaHelp, que deberás descargar

<https://central.sonatype.com/artifact/javafx.help/javafxhelp>

```
<dependency>  
  <groupId>javafx.help</groupId>  
  <artifactId>javafxhelp</artifactId>  
  <version>2.0.05</version>  
</dependency>
```


Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Para trabajar con **JavaHelp** además de descargar la librería, es necesario unos conocimientos básicos de HTML y XML:

Última versión estable de JavaHelp, que deberás descargar

<https://mvnrepository.com/artifact/javax.help/javahelp/2.0.05>

Biblioteca de ayuda para aplicaciones Java Swing

Sistema de Ayuda en una Aplicación

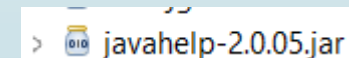
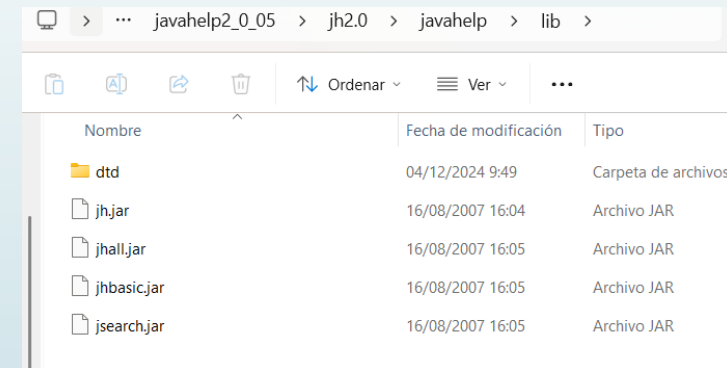
Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp en Aplicaciones SWING

Configura tu entorno

1.Descarga la biblioteca JavaHelp: Puedes obtenerla desde JavaHelp en SourceForge o del aulavirtual.

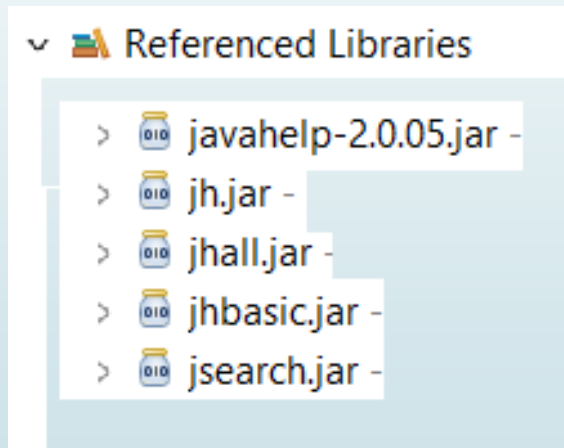
2.Incluye JavaHelp en tu proyecto: Agrega los archivos JAR (jh.jar, jhall.jar, jhbasic.jar y jsearch.jar) se encuentran dentro la carpeta lib del fichero que has descargado y descomprimido a tu proyecto. Añade



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Deberás tener añadidos al proyecto todos los jar, verás algo como así



Sistema de Ayuda en una Aplicación

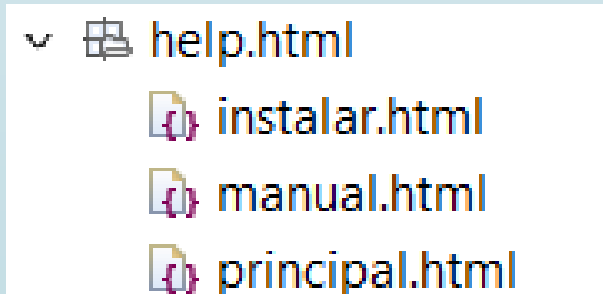
Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp

- 1. Crea una carpeta llamada help dentro del proyecto src/help**
- 2. Crea dentro de help una carpeta llamada html,** será donde guardes todos los documentos html.

Crea las páginas HTML que formarán parte de la ayuda. Por ejemplo:

- principal.html: Página principal.
- manual.html: Un tema específico.
- instalar.html: Otro tema.



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp

1. Archivos de contenido HTML de [principal.html](#)

```
<html>
  <head>
    <title>Ejemplo Integración Ayuda JavaHelp</title>
  </head>
  <body>
    <h1>Ejemplo de JavaHelp</h1>
    <p>Esta aplicacion sirve como ejemplo de uso de JavaHelp.</p>
    <p>Esta pagina es la ayuda principal. La aplicacion consta de dos ventanas,
    una <a href = "instalar.html">instalar</a> y la otra
    <a href="manual.html">manual</a>. Cada una de las ventanas tiene su propia
    pagina de ayuda.</p>
  </body>
</html>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp

1. Archivos de contenido HTML de [instalar.html](#)

```
1 <html>
2 <head>
3   <title>Instalar</title>
4 </head>
5 <body>
6   <h1>Instalar</h1>
7   <p>Aquí iría la ayuda de Instalar</p>
8 </body>
9 </html>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp

1. Archivos de contenido HTML de [manual.html](#)

```
1 <html>
2 <head>
3   <title>Manual</title>
4 </head>
5 <body>
6   <h1>Manual</h1>
7 </body>
8 </html>
```

Sistema de Ayuda en una Aplicación

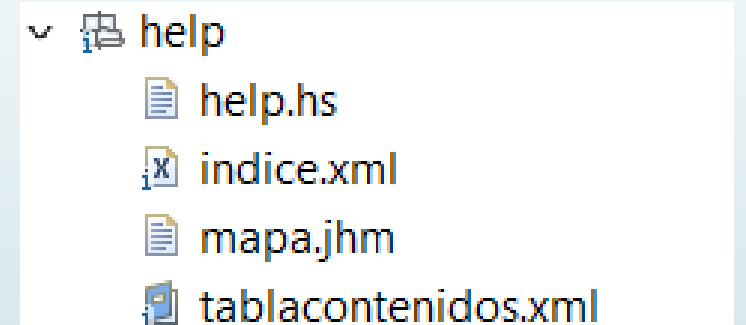
Sistema de Ayuda en la Aplicación SWING

Pasos para implementar un sistema de ayuda con JavaHelp

Crea los archivos necesarios

1. Archivos de configuración javahelp

- **help.hs**: Define la estructura del sistema de ayuda.
- **map.jhm**: Asocia identificadores con páginas específicas.
- **tablacontenidos.xml** Contenido de la Ayuda
- **indice.xml** definir la estructura y los enlaces entre los temas de ayuda y sus identificadores, tiene la función de mapear los identificadores de los temas de ayuda (ID) con las ubicaciones de los archivos HTML que contienen la información.



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Archivo **helpset.hs**

```
1<?xml version="1.0" encoding='UTF-8' ?>
2<helpset version="1.0">
3  <title>Sistema de Ayuda de la Aplicación</title>
4  <maps>
5    <!-- Pagina por defecto al mostrar la ayuda -->
6    <homeID>principal</homeID>
7    <!-- Que mapa deseamos -->
8    <mapref location="mapa.jhm"/>
9  </maps>
10 <!-- Las Vistas que deseamos mostrar en la ayuda -->
11 <view>
12   <name>Tabla Contenidos</name>
13   <label>Tabla de contenidos</label>
14   <type>javax.help.TOCView</type>
15   <data>tablacontenidos.xml</data>
16 </view>
17
18 <view>
19   <name>Indice</name>
20   <label>El indice</label>
21   <type>javax.help.IndexView</type>
22   <data>indice.xml</data>
23 </view>
24
25 <view>
26   <name>Buscar</name>
27   <label>Buscar</label>
28   <type>javax.help.SearchView</type>
29   <data engine="com.sun.java.help.search.DefaultSearchEngine">
30     JavaHelpSearch
31   </data>
32 </view>
33</helpset>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Archivo **índice.xml**

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <index version="1.0">
3     <indexitem text="Manual" target="manual"/>
4     <indexitem text="Instalar" target="instalar"/>
5 </index>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Archivo **map.jhm**

```
<?xml version='1.0' encoding='ISO-8859-1'?>
<map version="1.0">
  <mapID target="manual" url="html/manual.html" />
  <mapID target="instalar" url="html/instalar.html" />
  <mapID target="principal" url="html/principal.html" />
</map>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Archivo de **tablacontenido.xml**

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <toc version="1.0">
3   <!-- categoryclosedimage="FolderClosed" categoryopenimage="FolderOpened" topicimage="ItemIco"-->
4   <tocitem text="Principal" target="principal">
5     <tocitem text="Instalar" target="instalar"/>
6     <tocitem text="Manual" target="manual"/>
7   </tocitem>
8 </toc>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Integración en la aplicación

```
JMenuItem mntmIndice = new JMenuItem("Ayuda");  
menuBar.add(mntmIndice);
```

```
try {  
    // Carga el fichero de ayuda  
    File fichero = new File("src/help/help.hs");  
    URL hsURL = fichero.toURI().toURL();  
  
    // Crea el HelpSet y el HelpBroker  
    HelpSet helpset = new HelpSet(getClass().getClassLoader(), hsURL);  
    HelpBroker hb = helpset.createHelpBroker();  
  
    // Configurar ayuda para el menú Ayuda  
    hb.enableHelpOnButton(mntmIndice, "principal", helpset);  
    //Al pulsar F1 se muestra la ayuda  
  
    mntmIndice.setAccelerator(KeyStroke.getKeyStroke("F1"));  
  
} catch (Exception e) {  
    System.out.println("Error al cargar Ayuda");  
    e.printStackTrace();  
}
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Solo una última cosa mas

Sitúate en la carpeta help del proyecto.

Situado en ella ejecuta `java -jar path/jhindexer.jar html`

Ruta donde se encuentra la librería que has descargado `java\help2_0_05.zip\jh2.0\java\help\bin`

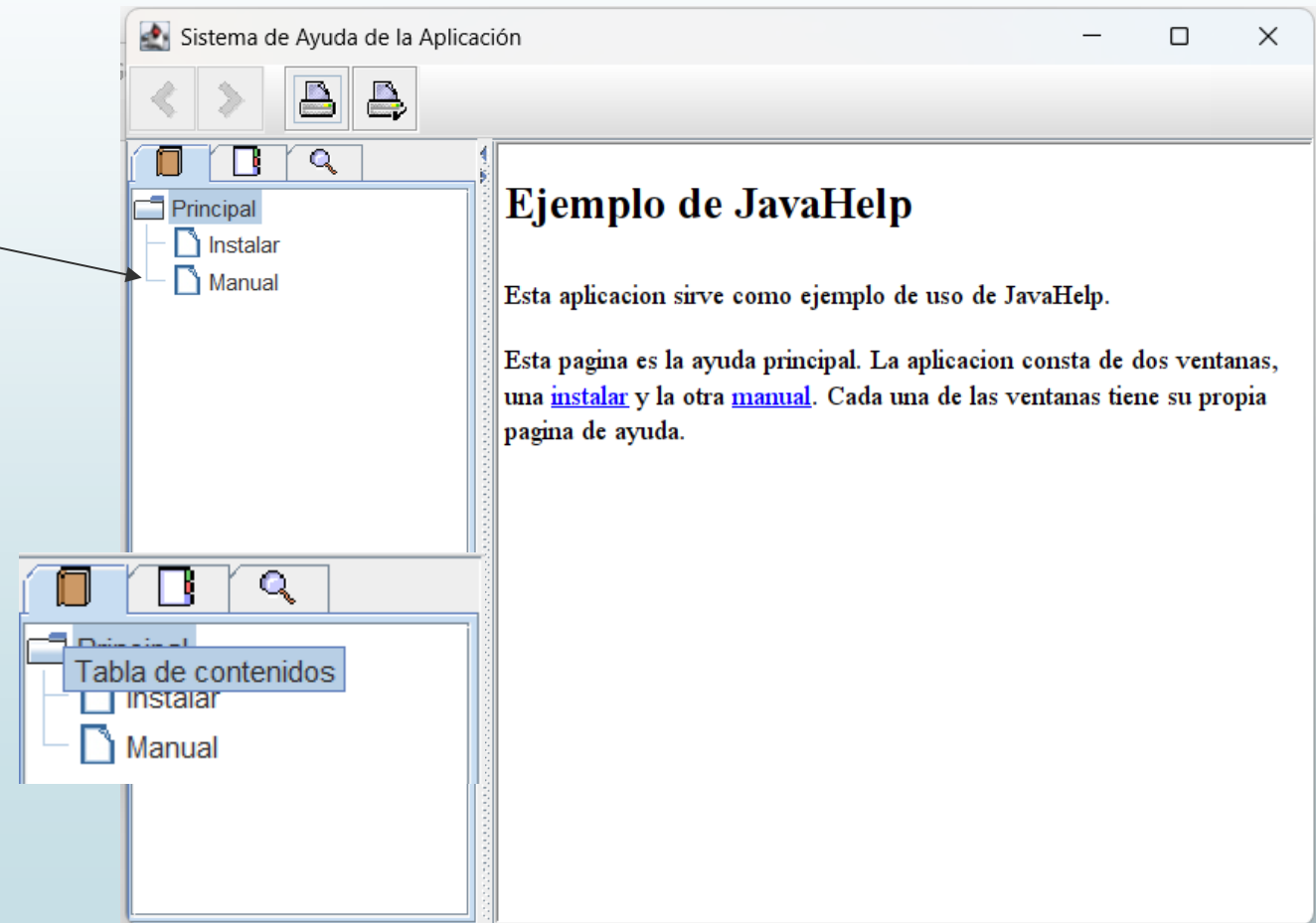
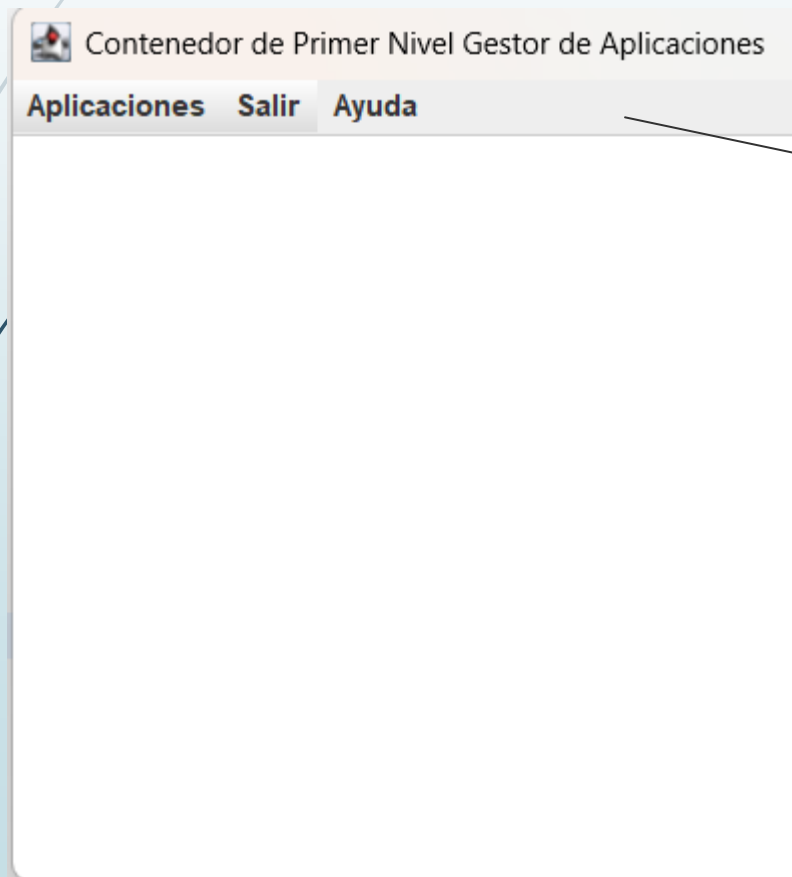
Una vez ejecutada la anterior instrucción se creará en el proyecto la carpeta **JavaHelpSearch**



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

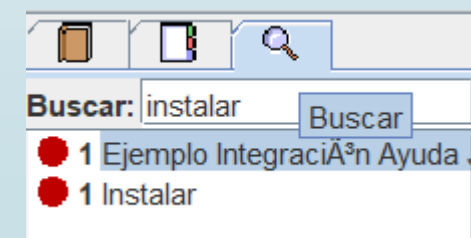
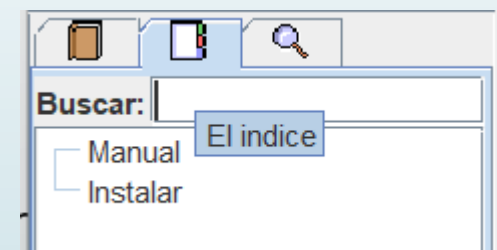
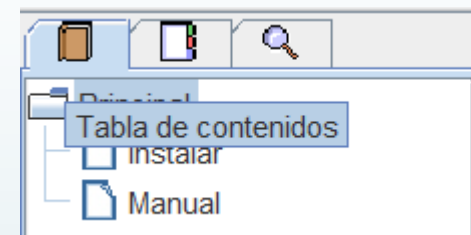
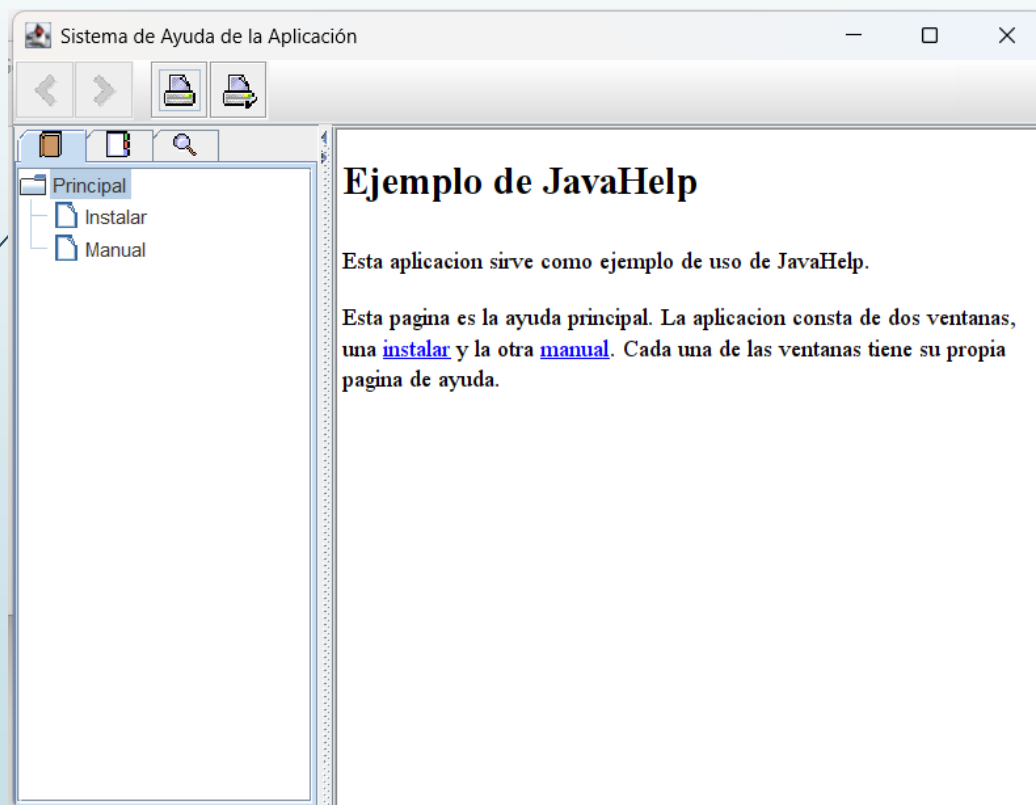
Demostración en la aplicación



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

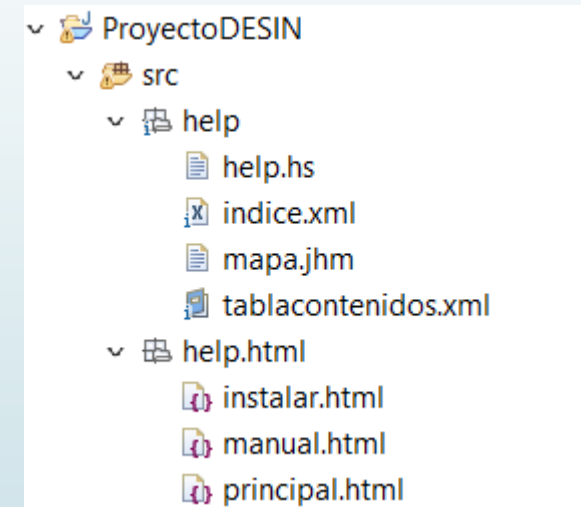
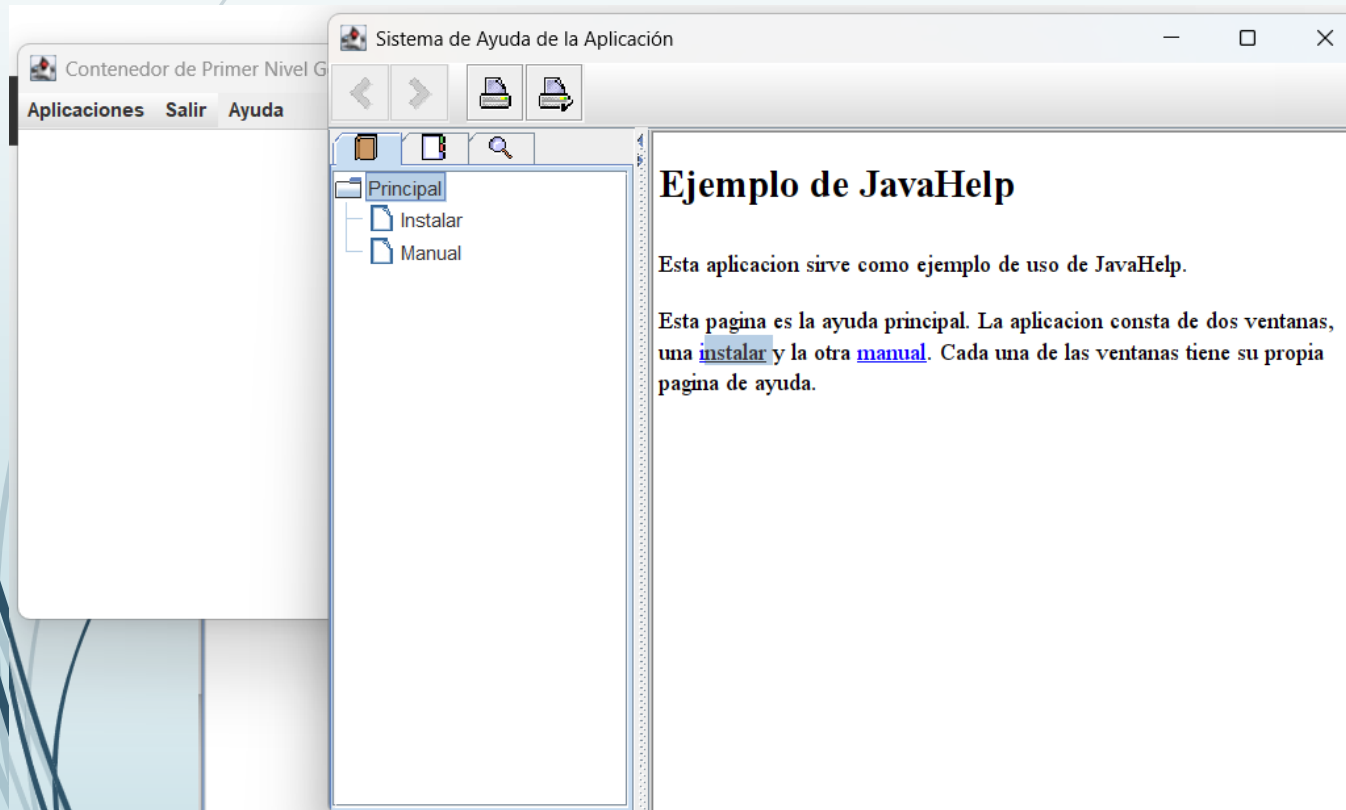
Demostración en la aplicación



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación SWING

Demostración en la aplicación tras pulsar F1



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

En las aplicaciones JavaFX, no existe una API como JavaHelp.

En JavaFX no existe un equivalente directo a **JavaHelp** de Swing, ya que esta biblioteca está diseñada específicamente para aplicaciones Swing y no se actualiza activamente.

Sin embargo, en JavaFX es posible implementar una funcionalidad de ayuda utilizando alternativas más modernas y flexibles.

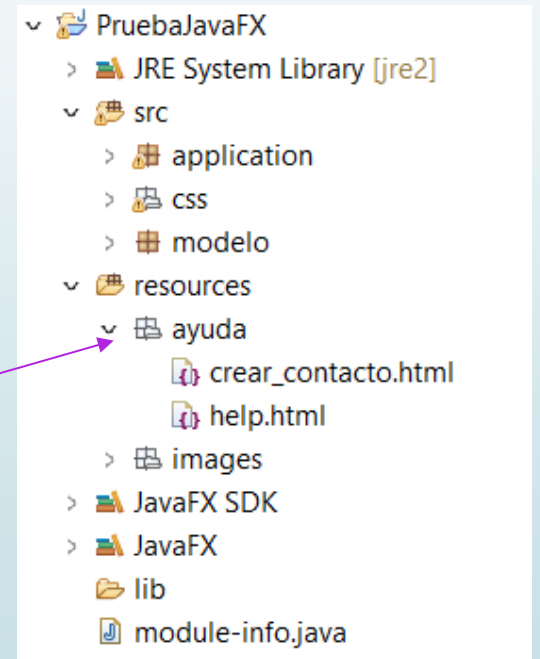
A continuación, se detalla cómo crear un sistema de ayuda en una aplicación **JavaFX**

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

1. Crea un Archivo HTML para la Ayuda: Guarda un archivo HTML con contenido estructurado de ayuda, por ejemplo, **help.html**. Crea en la carpeta **resources** una carpeta denominada ayuda, y dentro crea los ficheros necesarios para montar la ayuda.



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

2. Crea el contenido del fichero help.html. Por ejemplo



```
<html>
<head>
  <title>Ayuda</title>
  <meta charset="UTF-8">
</head>
<body>
  <h1>Bienvenido a la Ayuda de la Aplicación</h1>
  <p>Aquí encontrarás información sobre cómo usar la aplicación.</p>
  <a href="crear_contacto.html">Crear un Contacto Nuevo<a>
</body>
</html>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

2. Crea los ficheros necesarios para montar la ayuda. Por ejemplo



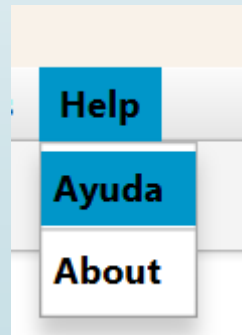
```
<html>
<head>
  <title>Ayuda</title>
  <meta charset="UTF-8">
</head>
<body>
  <h1>Crear un Contacto Nuevo</h1>
  <p>Aquí encontrarás información sobre cómo crear un nuevo contacto</p>
  <a href="help.html">Ir al Inicio<a>
</body>
</html>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

3. Crear una opción del menú para abrir la ayuda. Por ejemplo



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

4. Crear una opción del menú para abrir la ayuda (debes establecer el **action** en el MenuItem o Menu que establezcas para abrir la ayuda). Por ejemplo



```
<Menu text="Help">
  <items>
    <MenuItem fx:id="openHelp" onAction="#handleAyuda" text="Ayuda" />
    <MenuItem text="Online Manual" visible="false" />
    <SeparatorMenuItem />
    <MenuItem text="About" />
  </items>
</Menu>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Para crear un formato avanzado (como lo que JavaHelp permite), se requiere utilizar el componente **WebView** para mostrar contenido HTML que actúe como ayuda.

5.Crear el método que gestione la apertura de la ayuda, en este caso **handleAyuda**. Método que se implementará en el Controler que corresponda.

Para ello es necesario trabajar con el componente **WebView**

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

```
try {  
    // Crear un WebView para mostrar la ayuda  
    WebView webView = new WebView();  
  
    // Cargar el archivo HTML desde los recursos  
    String url = getClass().getResource("/ayuda/help.html").toExternalForm();  
    webView.getEngine().load(url);  
  
    // Crear un nuevo Stage para la ventana de ayuda  
    Stage helpStage = new Stage();  
    helpStage.setTitle("Ayuda");  
  
    // Crear una Scene con el WebView  
    Scene helpScene = new Scene(webView, 600, 600);  
  
    // Configurar la ventana  
    helpStage.setScene(helpScene);  
    // Bloquea la ventana principal mientras se muestra la ayuda  
    helpStage.initModality(Modality.APPLICATION_MODAL);  
    helpStage.setResizable(true);  
    // Mostrar la ventana de ayuda  
    helpStage.show();  
} catch (NullPointerException e) {  
    // Manejar el caso en que el archivo de ayuda no se encuentra  
    Alert alert = new Alert(Alert.AlertType.ERROR);  
    alert.setTitle("Error");  
    alert.setHeaderText("Archivo de Ayuda no encontrado");  
    alert.setContentText("Por favor, verifica que el archivo 'help.html' esté en la ruta '/ay");  
    alert.showAndWait();  
}
```

Se utiliza **WebView** para mostrar el contenido HTML de la ayuda.

El método **getEngine().load(url)** carga el archivo HTML desde los recursos del proyecto.

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**



Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

El módulo **javafx.web**, que contiene la clase `WebView`, no está incluido automáticamente en tu aplicación si estás usando JavaFX modular (Java 9 o posterior).

Si necesitas agregar explícitamente este módulo al classpath o modulepath.

Si usas Maven

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-web</artifactId>
  <version>21.0.0</version> <!-- Ojo!! Usa la versión de JavaFX que estes
utilizando -->
</dependency>
```

Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>23</version>
</dependency>
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-fxml</artifactId>
  <version>23</version>
</dependency>

<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-web</artifactId>
  <version>23</version>
</dependency>
```

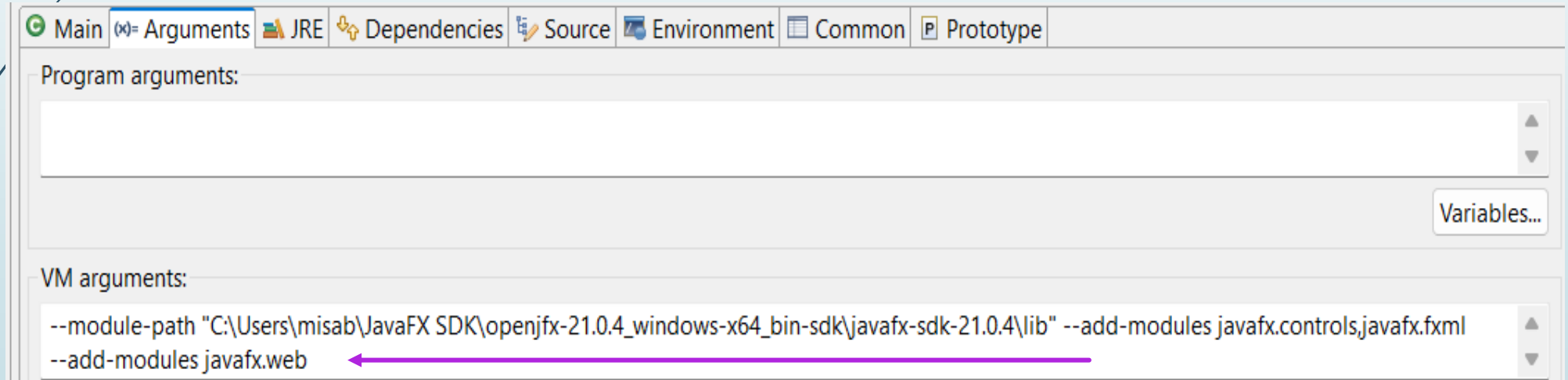
Sistema de Ayuda en una Aplicación

Sistema de Ayuda en la Aplicación **JAVAFX**

Si necesitas agregar explícitamente este módulo al classpath o modulepath.

Si estás ejecutando el programa desde la línea de comandos, debes incluir el módulo al iniciar la JVM:

--add-modules javafx.web



Si el proyecto es Maven no necesitas esto

Documentación Interna

Sistema de Ayuda en tu Aplicación

ACTIVIDAD A REALIZAR EN CLASE.

Tiempo de Realización 50 minutos

Instrucciones

1. **Modifica el fichero pom.xml para crear la ayuda**
2. **Modifica el menú para que puedas tener una opción de ayuda**
3. **Crea en la carpeta recursos los ficheros de ayuda correspondientes.**
4. **Crea en el controlador el método handleAyuda para mostrar la Ayuda.**
5. **Comprueba que la ayuda se muestra correctamente.**
6. **Entrega en la tarea habilitada en el AULA VIRTUAL un documento PDF donde se indiquen las evidencias de la realización de esta actividad. Deberá contener una Portada y cada uno de los pasos que se piden en las instrucciones de esta Actividad.**

Pautas para un Diseño Usable y Accesible

Webgrafía

Crear ficheros de ayuda en swing

<https://www.laurafolgado.es/desarrollointerfaces/2017/01/ejercicio-ut4e2-archivos-de-ayuda-con-javahelp/>

Ejemplo de uso de JavaHelp

<https://old.chuidiang.org/java/herramientas/javahelp/ejemplo-javahelp.php>

Desarrollo de Interfaces (Fernando Valdeón)

<https://interfaces.abrilcode.com/doku.php?id=start>

UT5-Documentación de Aplicaciones